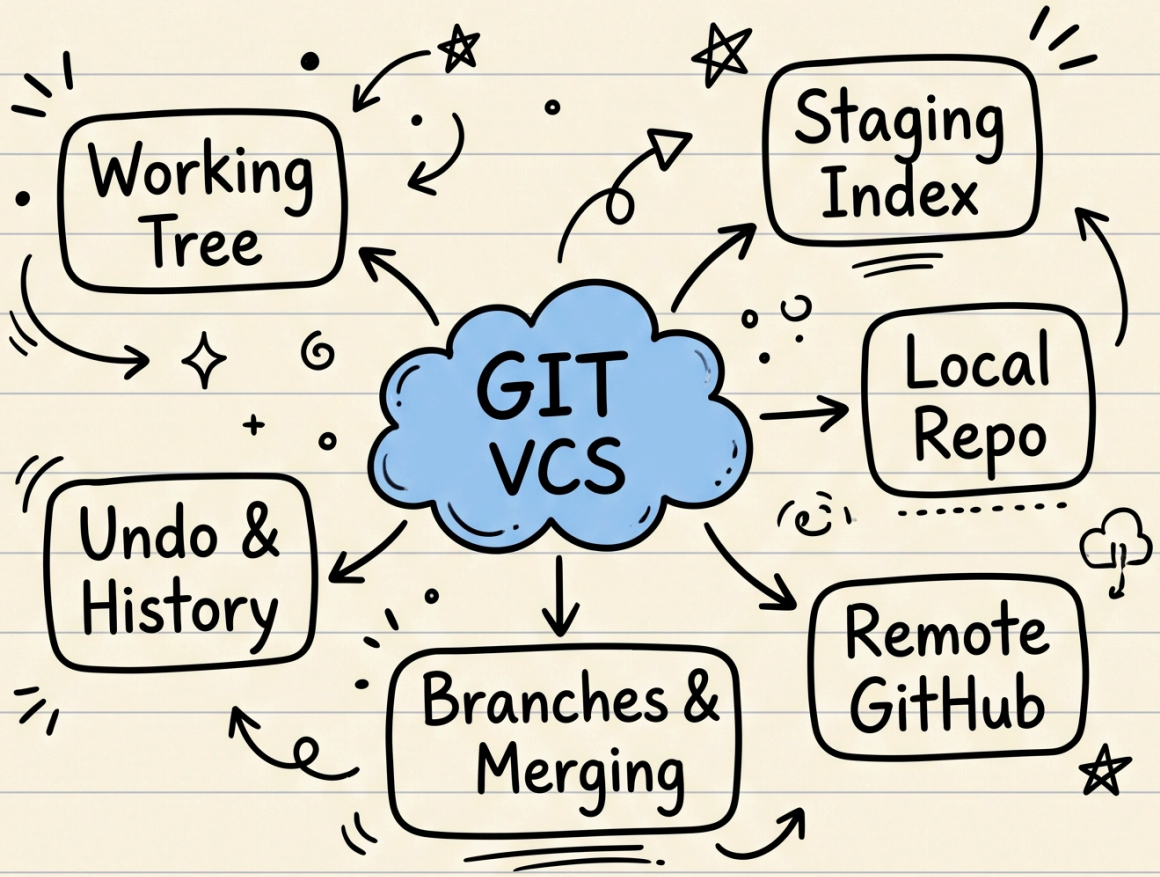
Version Control Basics

# GIT QUICK NOTES

BEGINNER TO INTERMEDIATE




SWIPE TO NEXT →

Snapshot | Stage | Commit | Branch

Version Control Basics

# ROADMAP

BEGINNER TO INTERMEDIATE

How to use these notes

Tools you will need

3. The Three Trees: Work, Stage & Commit

4. Branching, Merging & Moving HEAD

HEAD

5. Remotes, Syncing & Resolving Conflicts

6. Git Basics Cheat Sheet & Interview Guide

Mindset

Modern Git Standards

SWIPE TO NEXT →

Snapshot | Stage | Commit | Branch

Version Control Basics

BEGINNER TO INTERMEDIATE

# THE THREE TREES: WORK, STAGE & COMMIT

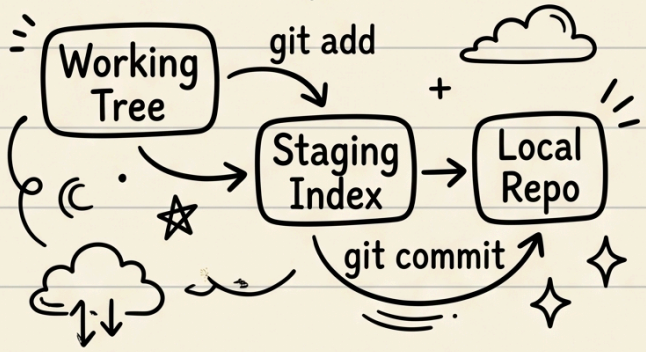
## 1. What is Git?

A distributed version control system to track changes in source code.

## 2. The Three Areas

- Working Tree (where you edit)
- Staging Index (where you prepare changes)
- Local Repo (where you save history)

## 3. The Lifecycle Flow




## 4. Core Staging Commands

| | |
|---|---|
| • git add <file> | Stage changes |
| • git status | Check status |
| • git reset | Unstage) |

## 5. First Commit Example

```
$ git add index.html
$ git commit -m "initial commit"
[main (root-commit) 12345]
initial commit
```

## 6. Common Mistakes & Fixes

Forgot a file?
Use `git commit --amend`.

Accidentally staged? Use `git restore --staged <file>`.

## 7. Remember!

Commits are snapshots, not just differences!

Real-world use cases: Feature branches & collaboration.

Interview tip: Know the difference between staging and committing.

Pro tip: Commit often, but push only when ready!

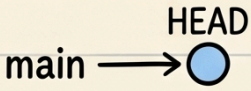
Version Control Basics

# Branching, Merging & Moving HEAD

BEGINNER TO INTERMEDIATE

## 1 What is a Branch?

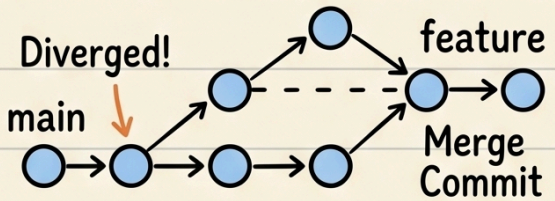
A branch is a lightweight pointer to a commit (e.g., HEAD -> main -> commit)




## 2 Modern Navigation

| Command | Behavior |
|---|---|
| git switch <branch> | Switch branches |
| git checkout -b <new> | Create & switch |
| git restore <file> | Discard changes |

## 3 Branch Divergence & Merge




## 4 Merge Strategies

- Fast-Forward (linear history, just moves pointer)
- 3-Way Merge (creates a new merge commit)

## 5 Feature Branch Workflow

```
git checkout -b feat/login
git commit -am 'add login page'
git checkout main
git merge feat/login
```

Output
...

## 6 Common Traps & Fixes

1. Detached HEAD:
   (Checked out a commit instead of a branch!
   Fix: git switch <branch>)
2. Merge Conflicts:
   (Same line changed!
   Fix: Resolve manually, git add, git commit)

## 7 Remember

A branch is just a pointer! HEAD is where YOU are currently looking!

SWIPE TO NEXT →

Real-world use cases: Feature-driven development keeps the main branch always deployable.

Interview tip: Fast-forward only happens if no new commits exist on the base branch.

Pro tip: Use `git log --graph --oneline` to view branch history visually.



# GIT BASICS

BEGINNER TO INTERMEDIATE

## CHEAT SHEET & INTERVIEW GUIDE

1. **Snapshot Model**
   - Captures state

2. **Three States**
   - Modified
   - Staged
   - Committed

3. **Branch Pointers**
   - pointer to commit

4. **switch & restore**
   - switch branch/ restore files

5. **Merging Types**
   - Fast-forward & 3-way

6. **Rebase vs Merge**
   - Linear history vs merge commit

7. **Remote Sync**
   - fetch, pull, push

8. **Stashing Work**
   - git stash

9. **Undoing Mistakes**
   - reset, revert

10. **Conflict Markers**
    `<<<<<<< HEAD`
    `======= >>>>>>>`

11. **The Reflog**
    `git reflog`

12. **Git Aliases**
    custom shortcuts

### TOP 10 INTERVIEW QUESTIONS

1. Merge vs Rebase?
2. What is HEAD?
3. Fetch vs Pull?
...

### THE ULTIMATE GIT FORMULA

Code + Stage + Commit = History

Cheat Sheet | QA Guide | Commands